

Analysis: Increasing Speed Limits

This was the hard problem for round 1C, with 398 people solving the small i/o set and 49 people solving the large i/o set during the contest.

The problem asks us to count the number of strictly increasing subsequences of the original sequence. This lends itself nicely to a dynamic programming solution for the small input.

Let $f(x)$ be the number of strictly increasing subsequences that start with node x and $S[x]$ be the value of the sequence at position x . At any node x we can either end the subsequence at x or connect it to any of the following nodes that are strictly greater than node x . Therefore, you could do something like this to find out how many strictly increasing subsequences start at each index.

```
f(x) = 1
for i = n to 1
    f(i) = 1
    for j = i + 1 to n
        if S[i] < S[j]
            f(i) = f(i) + f(j)
```

After doing that, solving the original problem is just a matter of summing the number of ways you make a strictly increasing subsequence starting at each position.

The key to solving the large i/o set is to realize we can make the inner loop run much faster. What we really want to do is sum all of the previous $f(j)$ s for which $S[j] > S[i]$. This is a classic tree problem. There are a number of tree data structures you could use including a binary indexed tree, a segment tree, or a binary search tree, to name a few, that will allow you to solve the problem in $O(n \log n)$ time per case.

The first two types of trees are fairly easy to implement although they add a complication since they use an amount of memory proportional to the maximum value in the sequence. Fortunately this can be worked around by normalizing S to contain only values between 0 and $n-1$ without changing the $S[i] < S[j]$ property for any i and j . This can be done by transforming $S[i]$ to be the first occurrence of $S[i]$ in a sorted list of S .

Here is an implementation of these ideas in C++.

```
#define MAXN (1<<20)
int sum_bit[MAXN];

int sum_bit_get(int x)
{
    int ret = 0;
    for(int i = x | MAXN; i < 2 * MAXN; i += i & -i)
        ret = (ret + sum_bit[i ^ MAXN]) % 1000000007;
    return ret;
}

void sum_bit_add(int x, int v)
{

```

```

    for(int i = x | MAXN; i; i &= i - 1)
        sum_bit[i ^ MAXN] = (sum_bit[i ^ MAXN] + v) % 1000000007;
}

int S[1000000];
int S2[1000000];

int main()
{
    int T; cin >> T;
    long long X, Y, Z, A[100];
    for(int t = 1; t <= T; t++) {
        int n, m; cin >> n >> m >> X >> Y >> Z;
        for(int i = 0; i < m; i++) cin >> A[i];

        // Generate S
        for(int i = 0; i < n; i++) {
            S[i] = A[i % m];
            A[i % m] = (X * A[i % m] + Y * (i + 1)) % Z;
        }

        // Normalize S
        memcpy(S2, S, sizeof(S));
        sort(S2, S2+n);
        for(int i = 0; i < n; i++)
            S[i] = lower_bound(S2, S2+n, S[i]) - S2;

        // Calculate f(i) and sum them in to result.
        int result = 0;
        memset(sum_bit, 0, sizeof(sum_bit));
        for(int i = n - 1; i >= 0; i--) {
            int add = 1 + sum_bit_get(S[i] + 1);
            sum_bit_add(S[i], add);
            result = (result + add) % 1000000007;
        }

        cout << "Case #" << t << ": " << result << endl;
    }
    return 0;
}

```

In addition to trees, there was a "hacker's" $O(n \sqrt{n})$ solution. The idea behind it is similar to the idea mentioned in solution 1 of the Mousetrap editorial (from Round 1B) where, in addition to maintaining the value at each position, we maintain sums for ranges of length $\sqrt{N}$. These tables can be maintained in constant time and take $\sqrt{n}$ time to query.

More Information:

[Binary Indexed Tree](#)

Analysis: Text Messaging Outrage

This was the one of the easiest problems of Round 1. We simply need to fill the cell phone keyboard greedily. We put the **K** most frequent letters in the first positions of the **K** keys, the next **K** most frequent letters in the second positions, and so on. Any optimal solution will have this structure because if it does not, then it can be improved by swapping a more frequent character from a higher indexed position of some key with a less frequent character from a lower indexed position, thus decreasing the number of key presses.

Here is code that implements this solution:

```
long long A[1000];

int main() {
    int N;
    cin >> N;
    for(int t = 1; t <= N; t++) {
        long long result = 0;
        int P, K, L;
        cin >> P >> K >> L;
        for(int i = 0; i < L; i++) cin >> A[i];
        sort(A, A+L);
        reverse(A, A+L);

        for(int i = 0; i < L; i++)
            result += (1 + i / K) * A[i];

        cout << "Case #" << t << ": " << result << endl;
    }
}
```

Analysis: Ugly Numbers

As the problem clearly states, there are 3^{D-1} ways you need to consider. When **D** is no larger than 13, as it is in the small dataset, one may simply generate all the possible combinations and calculate each of them. However, with **D** as large as 40, brute force is clearly too slow.

As you might expect from a programming contest, the magic term is [dynamic programming](#). This is the first problem in Google Code Jam 2008 that falls into the standard dynamic programming (DP) category; and one can be assured that there are more to come.

As in any DP problem, the first task is to structure the problem in a good way so there are not too many numbers to compute, and each number can be computed easily from previous ones. If you observe the solutions of the top scorers in this sub-round, you will see, although their algorithms vary to some degree, they all contain the following magic number

$$2 \cdot 3 \cdot 5 \cdot 7 = 210.$$

Let us say we have two numbers, x and y , knowing the ugliness of x and y is not enough to decide whether $x + y$ and $x - y$ are ugly. On the other hand, we do not need the exact value of x and y . Knowing $x \% 210$ and $y \% 210$ is enough for us to decide if, say, $x + y$ is ugly or not.

For those who prefer mathematical terms, we are using the [Chinese remainder theorem](#). Our problem can be viewed as arithmetics on the cyclic group $(\mathbb{Z}_{210}, +)$.

So, let us outline the most central step of our dynamic programming solution. We want to compute

```
dyn[i][x] := number of ways we get an expression evaluating
            to x (mod 210) if we only consider the first i
            characters of the string. (*)
```

So, we have only $40 \cdot 210$ numbers to consider. For each $\text{dyn}[i][x]$, we try all possible positions for inserting the last '+' or '-' sign. If the last sign was a '+' before position j ($j < i$), and the number formed by digits from position j to position i is d , then we want to know $\text{dyn}[j-1][(x-d)\%210]$. On the other hand, if the sign inserted was a '-', then we want to look at $\text{dyn}[j-1][(x+d)\%210]$.

Here is a masterful implementation from our own Derek Kisman. His complete C++ solution follows the idea above, with a little twist and some nice programming tricks.

```
#define MOD (2*3*5*7)
string s;
long long dyn[41][MOD];

main() {
    int N, prob=1;
    for (cin >> N; N--;) {
        cin >> s;
        memset(dyn, 0, sizeof(dyn));
        dyn[0][0] = 1;
        for (int i = 0; i < s.size(); i++)
            for (int sgn = (i==0) ? 1 : -1; sgn <= 1; sgn += 2) {
                int cur = 0;
                for (int j = i; j < s.size(); j++) {
```

```

        cur = (cur*10 + s[j]-'0')%MOD;
        for (int x = 0; x < MOD; x++)
            dyn[j+1][(x+sgn*cur+MOD)%MOD] += dyn[i][x];
    }
}
long long ret = 0;
for (int x = 0; x < MOD; x++)
    if (x%2 == 0 || x%3 == 0 || x%5 == 0 || x%7 == 0)
        ret += dyn[s.size()][x];
cout << "Case #" << prob++ << ": " << ret << endl;
}
}

```

More information:

[Dynamic programming](#) - [Chinese remainder theorem](#)